# MemDB2 C DLL Benchmark Results

mdb2bench, 10 November 2026 · x64 Release DLL · 1 million records · 4 readers + 4 writers

**0.57–0.64 ms**
single write transaction (insert, delete), isolated

**36–98 µs**
single read transaction, isolated, depending on access mode

**6.5k txn/s**
mixed 4 + 4 workload, all transactions READ_WRITE_SHARED

**-4%**
throughput with async checkpoints running; worst case 48 ms → 884 ms

## Summary

All figures come from an old Ivy Bridge laptop, with every call going through the C DLL. Expect modern hardware, and native Delphi use, to be considerably faster (see Caveats).

- **Writes cost about 0.6 ms each, updates about twice that.** Across the three runs, insert, delete by ID and delete by expiry take 0.57–0.64 ms (1.23–1.40 million CPU cycles) as single transactions. State transition and recycle, which update a row in place, take 1.08–1.22 ms (2.36–2.62 million cycles).
- **Reads cost 36–51 µs in READ_SHARED mode, and about twice that in READ_WRITE_SHARED** (79–98 µs).
- **Under the mixed workload, the reader access mode decides who gets the throughput.** With READ_SHARED readers the database ran 11,584 txn/s, but writers got only 1,537/s. With all transactions READ_WRITE_SHARED, writers more than doubled to 3,443/s, readers fell to 3,069/s, and the total was 6,512/s.
- **The tail is long and fairly flat.** p99.99 is 15–25 ms for every write, and for reads in READ_WRITE_SHARED mode, almost regardless of the operation's own cost. Reads in READ_SHARED mode stay below 0.2 ms at p99.99.
- **Asynchronous checkpoints mostly leave the workload alone, except at the extreme tail.** Throughput fell 4.4% and the median was unchanged. p99.9 rose 3.3× and the worst case 18×, to 884 ms. Each checkpoint took about 13 s, against 3.2 s for a synchronous checkpoint of the same data.

## Test setup

| | |
|---|---|
| Machine | Laptop, Intel Core i7-3632QM (Ivy Bridge, 2012), 2.2 GHz, 4 cores / 8 threads; invariant TSC (2.195 GHz) |
| Database | MemDB2 C DLL, x64 Release, without the benchmark allocation hooks. All calls go through the C API shim (MemDB2CApi), not native Delphi |
| Data | 1,000,000 rows of about 128 bytes (Id, Expiry, State, Payload blob); unique index on Id, index on Expiry |
| Transactions | Writers READ_WRITE_SHARED, lazy write journal, repeatable read isolation. Readers READ_SHARED (default run) or READ_WRITE_SHARED (rws runs) |
| Isolated phase | Each of 7 operations alone on one thread, 200,000 transactions each |
| Mixed phase | 4 reader + 4 writer threads, unpaced, 30 s after a 5 s warm-up. Per insert: delete by ID p = 0.5, state transition p = 0.25, recycle p = 0.1; expiry sweep every 500 µs. Readers: find by ID, with find earliest every 10th read |
| Runs | `--records 1000000` (default); … `--reader-mode rw` (rws); … `--reader-mode rw --ckpt async` (rws-ckpt, back-to-back asynchronous checkpoints during the mixed phase) |

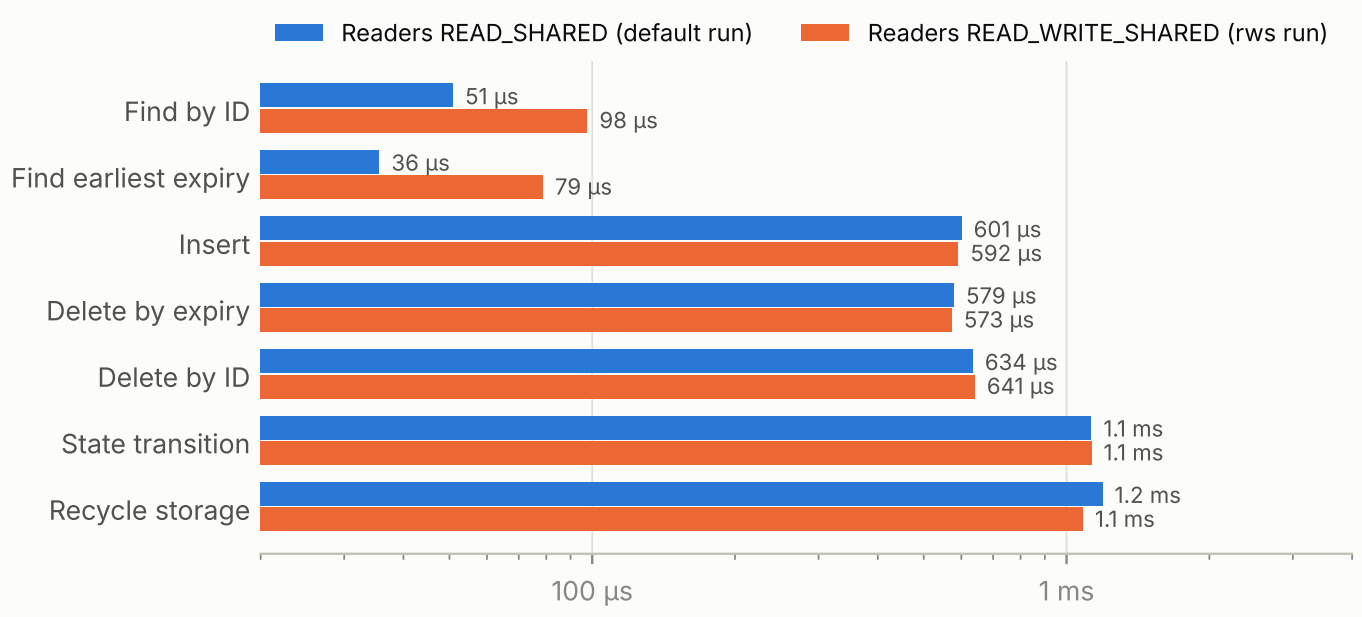
Loading the 1M rows took 240–261 s (3,800–4,200 rows/s, in transactions of 10,000 rows) and used about 2,130 bytes of process memory per row, roughly 17 times the logical row size. A synchronous checkpoint of the loaded data took 3.2–3.3 s.

## Isolated operations

Each operation runs as a complete transaction on one thread, with nothing else running, so these are the base costs of the engine and the C API.

**Figure 1. Mean latency per transaction, isolated**

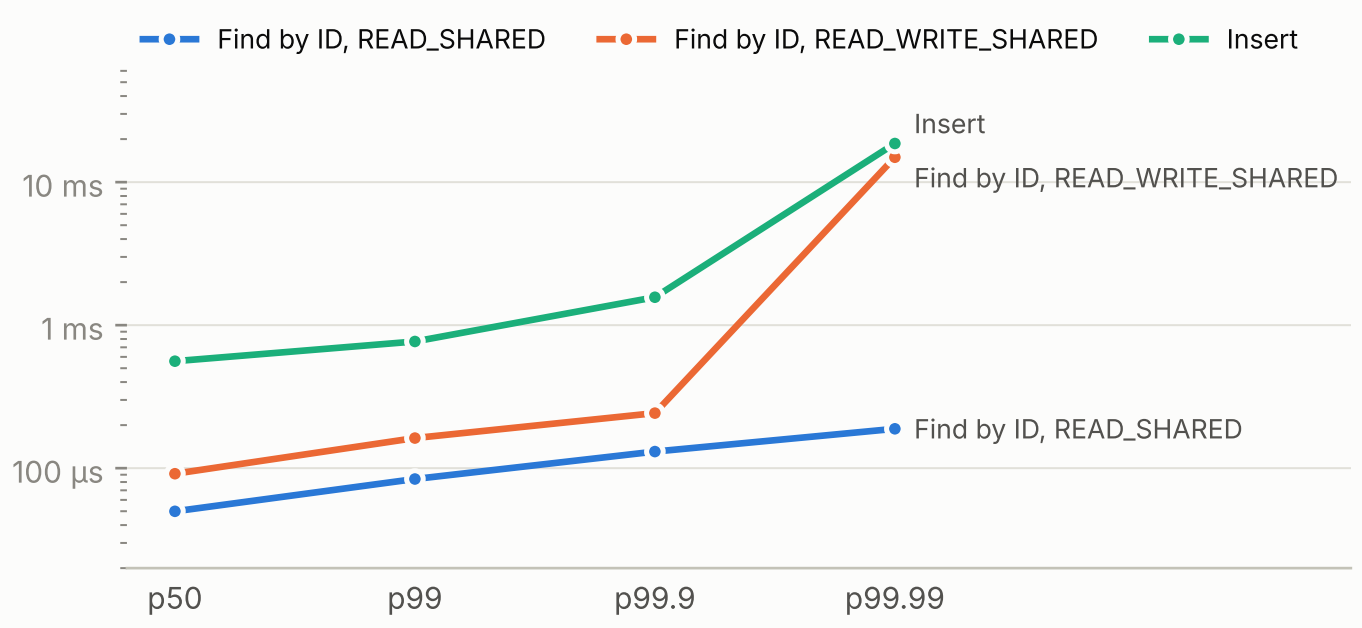
Logarithmic scale. Only reads depend on the readers' access mode; the writes ran identically in both runs.




Updates in place (state transition, recycle) cost about twice an insert or delete. Reads in READ_WRITE_SHARED mode cost about twice as much as in READ_SHARED: 207,000 against 112,000 CPU cycles for a find by ID.

**Figure 2. Latency percentiles, isolated**

Logarithmic scale. Find by ID in both access modes, and insert.

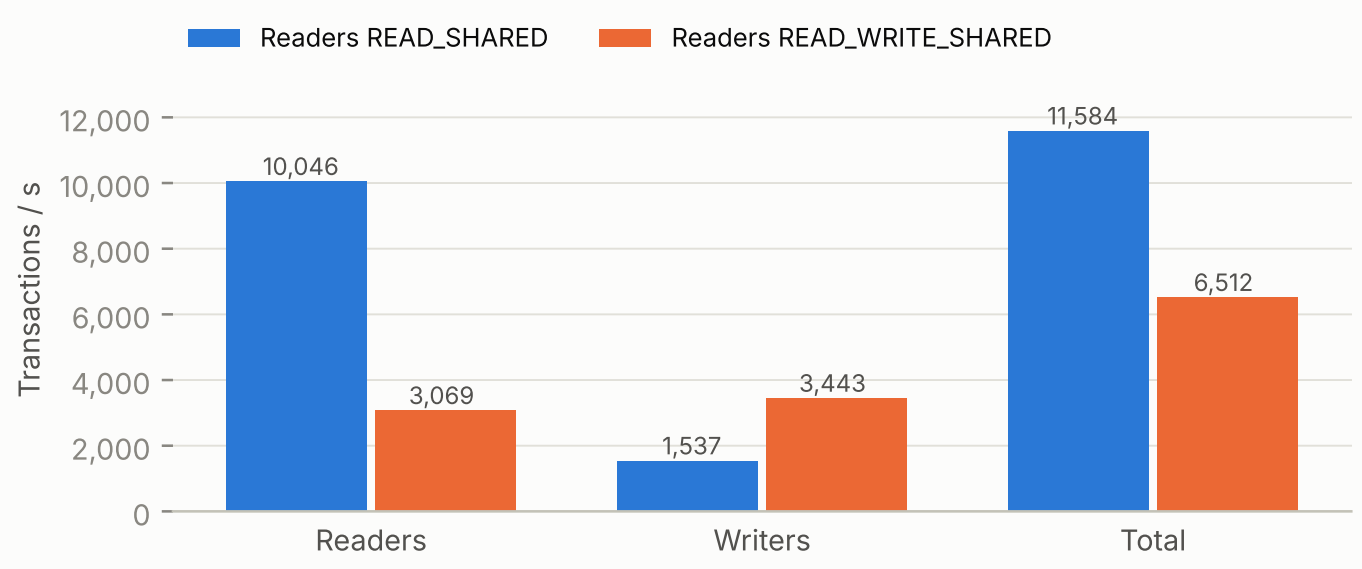



Up to p99.9, latency tracks each operation's own cost. At p99.99 every write, and every read in READ_WRITE_SHARED mode, lands at 15–25 ms; reads in READ_SHARED mode stay at 0.17–0.19 ms. Something periodic, of roughly constant length, stalls transactions in the write-capable modes. That's the first thing to profile. Single-run maxima ranged up to 3.2 s and weren't consistent between runs, so they're left out of this comparison.

## Mixed workload

**Figure 3. Throughput by role, mixed workload**

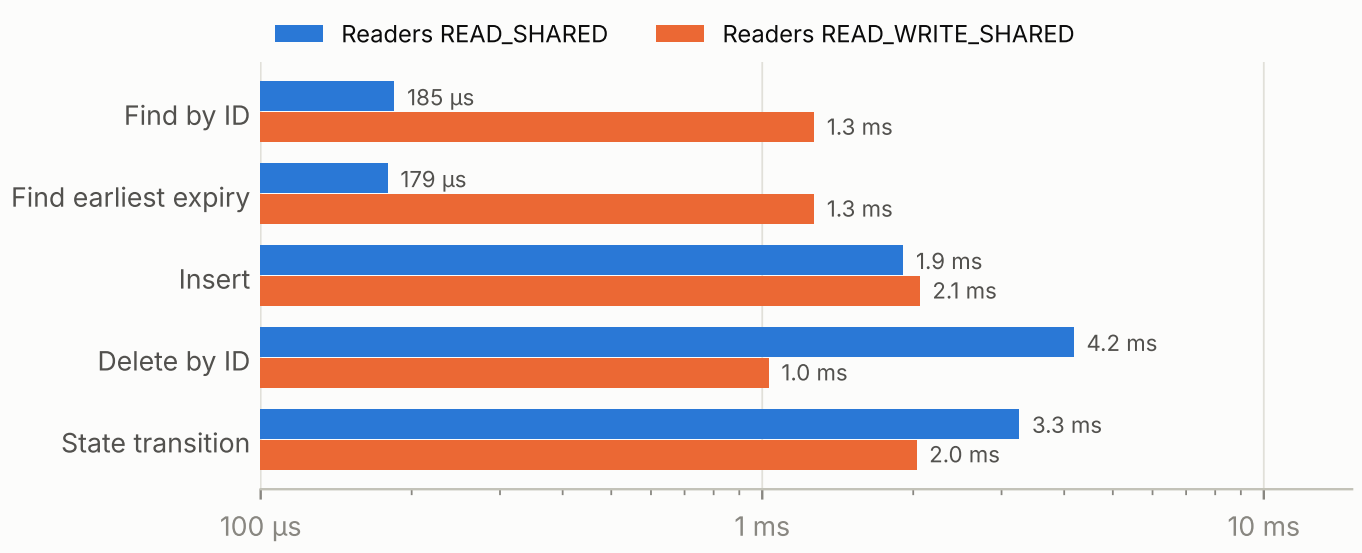
Transactions per second over the 30 s measurement.




READ_SHARED and READ_WRITE_SHARED transactions take the database's read/write lock in different modes, so they run in alternating batches. With readers in READ_SHARED, reads dominate: 10,046 reads/s against 1,537 writes/s. With every transaction in READ_WRITE_SHARED there is one mode, and the split evens out.

**Figure 4. Median latency per operation, mixed workload**

Logarithmic scale. Delete by expiry and recycle are left out: they found nothing to do (see Caveats).



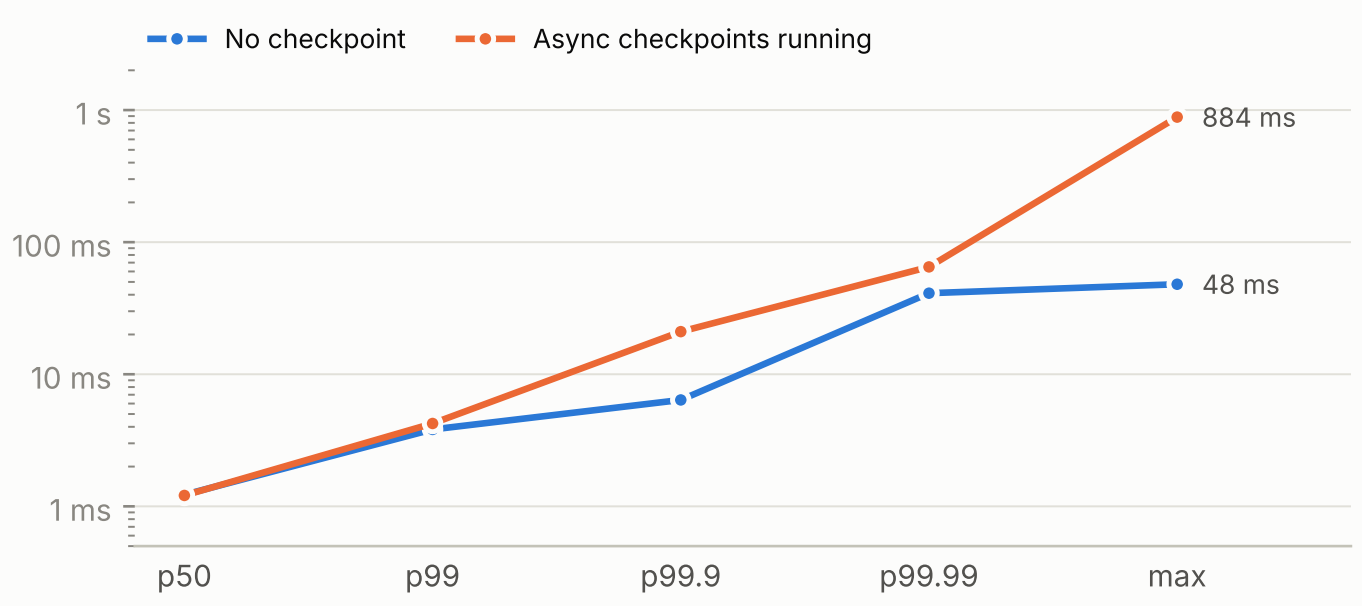

The same effect shows in the medians. Reads are about 7 times faster with READ_SHARED readers (185 µs against 1.27 ms), while writers wait longer: a delete by ID's median is 4.2 ms against 1.0 ms. Total CPU per transaction was 437,757 cycles (READ_SHARED) and 1,047,605 (READ_WRITE_SHARED), reflecting the different mix. No transaction hit a concurrency clash in any run.

## Asynchronous checkpoints

The rws-ckpt run repeats the rws run with a separate thread writing asynchronous checkpoints back to back during the mixed phase. A checkpoint was running for 84% of the measured time, so every transaction overlapped one, and the comparison is between the two runs.

**Figure 5. Latency percentiles, all mixed-phase transactions**

Logarithmic scale. rws (no checkpoint) against rws-ckpt (asynchronous checkpoints running).




| Measure | No checkpoint | Async checkpoints | Change |
|---|---|---|---|
| Throughput (txn/s) | 6,512 | 6,224 | -4% |
| Inserts/s | 1,268 | 1,220 | -4% |
| p50 | 1.22 ms | 1.21 ms | -1% |
| p99 | 3.82 ms | 4.24 ms | +11% |
| p99.9 | 6.39 ms | 21 ms | ×3.3 |
| p99.99 | 41 ms | 65 ms | ×1.6 |
| Max | 48 ms | 884 ms | ×18 |
| Process CPU cycles per transaction | 1,047,605 | 1,436,227 | +37% |

Writers keep committing during an asynchronous checkpoint, as designed: the median is unchanged and throughput dropped only 4.4%. The cost is in the tail. Insert, find and delete by ID each had worst cases of 0.83–0.88 s, which suggests a short, complete stall during each checkpoint, most likely while the snapshot is taken under the commit lock.

Each checkpoint took 13.1 s on average (12.8 s median), about four times a synchronous checkpoint of the same data (3.2 s). Two complete checkpoints used 44.9 billion cycles on the checkpoint thread, about 10 s of CPU each, so the checkpoint itself is largely CPU-bound rather than waiting on the disk.

## Caveats

- **Old hardware.** The tests ran on a 2012 Ivy Bridge laptop (Core i7-3632QM, 2.2 GHz). Absolute figures are unlikely to be representative of modern hardware; the comparisons between configurations are more meaningful than the raw numbers.
- **Every call goes through the C DLL shim.** Each transaction makes about 6–10 C API calls. Each call validates and looks up its handle in a handle table guarded by a single lock, and fields are named by string on every read and write. The shim is likely to be considerably slower than using MemDB2 natively from Delphi, and the shared lock may also add contention in the mixed workload.
- **Expiry processing wasn't exercised in the mixed phase.** Records expire 1,386,294 inserts after their own insert, but only 22,000–51,000 inserts happened during each mixed phase, so nothing expired. Every delete-by-expiry and recycle transaction in that phase was a miss: an empty probe of the expiry index, then a rollback. A shorter `--lifetime` is needed to measure expiry under load. In the isolated phase, expired rows were available and those transactions did real work.
- **Misses are included in the latencies.** 43% of mixed-phase finds by ID, and 43% of deletes by ID, targeted rows already deleted.
- **One run per configuration.** The isolated phase runs identically in the rws and rws-ckpt runs, since checkpoints only run in the mixed phase. Their means agree within 3%, except state transition (+8%), which gives a rough idea of run-to-run variation.
- **The READ_SHARED run is from the execution log only.** It wasn't written to `results.csv`.
- **Not covered here:** the baseline engine (plain data structures, to measure MemDB2's overhead), flush-per-commit journalling, x86, the specified 5 million rows, and allocations per transaction (the DLL was built without the counting hooks).

## Appendix: data

### Isolated phase, default run (readers READ_SHARED)

| Operation | txn/s | mean | p50 | p99 | p99.9 | p99.99 | max | thread cycles | process cycles |
|---|---|---|---|---|---|---|---|---|---|
| Insert | 1,700 | 601 µs | 567 µs | 888 µs | 1.28 ms | 18 ms | 2.51 s | 1,274k | 1,288k |
| Find by ID | 19,600 | 51 µs | 50 µs | 84 µs | 131 µs | 188 µs | 381 µs | 112k | 112k |
| Find earliest expiry | 28,000 | 36 µs | 35 µs | 63 µs | 103 µs | 166 µs | 359 µs | 78k | 78k |
| Delete by ID | 1,600 | 634 µs | 627 µs | 806 µs | 1.04 ms | 19 ms | 39 ms | 1,381k | 1,389k |
| Delete by expiry | 1,700 | 579 µs | 560 µs | 754 µs | 1.25 ms | 19 ms | 820 ms | 1,251k | 1,259k |
| State transition | 900 | 1.13 ms | 1.10 ms | 1.46 ms | 2.60 ms | 23 ms | 41 ms | 2,457k | 2,468k |
| Recycle storage | 800 | 1.19 ms | 1.15 ms | 1.79 ms | 3.34 ms | 25 ms | 1.56 s | 2,573k | 2,583k |

### Isolated phase, rws run (readers READ_WRITE_SHARED)

| Operation | txn/s | mean | p50 | p99 | p99.9 | p99.99 | max | thread cycles | process cycles |
|---|---|---|---|---|---|---|---|---|---|
| Insert | 1,700 | 592 µs | 560 µs | 769 µs | 1.57 ms | 19 ms | 3.15 s | 1,247k | 1,261k |
| Find by ID | 10,200 | 98 µs | 91 µs | 162 µs | 243 µs | 15 ms | 19 ms | 207k | 213k |
| Find earliest expiry | 12,700 | 79 µs | 70 µs | 125 µs | 188 µs | 15 ms | 324 ms | 159k | 165k |
| Delete by ID | 1,600 | 641 µs | 634 µs | 814 µs | 1.10 ms | 20 ms | 31 ms | 1,394k | 1,403k |
| Delete by expiry | 1,700 | 573 µs | 560 µs | 731 µs | 1.07 ms | 19 ms | 22 ms | 1,246k | 1,255k |
| State transition | 900 | 1.13 ms | 1.10 ms | 1.45 ms | 2.39 ms | 24 ms | 47 ms | 2,459k | 2,469k |
| Recycle storage | 900 | 1.08 ms | 1.06 ms | 1.39 ms | 1.84 ms | 22 ms | 24 ms | 2,362k | 2,372k |

### Isolated phase, rws-ckpt run (no checkpoints in this phase)

| Operation | txn/s | mean | p50 | p99 | p99.9 | p99.99 | max | thread cycles | process cycles |
|---|---|---|---|---|---|---|---|---|---|
| Insert | 1,700 | 573 µs | 560 µs | 731 µs | 1.30 ms | 19 ms | 958 ms | 1,233k | 1,246k |
| Find by ID | 10,300 | 97 µs | 90 µs | 157 µs | 231 µs | 15 ms | 111 ms | 204k | 210k |
| Find earliest expiry | 12,400 | 81 µs | 69 µs | 140 µs | 215 µs | 16 ms | 384 ms | 159k | 165k |
| Delete by ID | 1,600 | 643 µs | 627 µs | 881 µs | 1.37 ms | 19 ms | 95 ms | 1,396k | 1,404k |
| Delete by expiry | 1,700 | 575 µs | 560 µs | 717 µs | 933 µs | 18 ms | 509 ms | 1,245k | 1,253k |
| State transition | 800 | 1.22 ms | 1.16 ms | 2.21 ms | 4.36 ms | 24 ms | 967 ms | 2,621k | 2,631k |
| Recycle storage | 900 | 1.08 ms | 1.06 ms | 1.39 ms | 2.06 ms | 22 ms | 37 ms | 2,364k | 2,374k |

Cycles are CPU cycles per transaction: thread = the calling thread; process = the whole process, including MemDB2's journal thread.

### Mixed phase, default run (readers READ_SHARED)

| Operation | txns | txn/s | mean | p50 | p99 | p99.9 | p99.99 | max | misses |
|---|---|---|---|---|---|---|---|---|---|
| Insert | 17,546 | 600 | 2.43 ms | 1.91 ms | 7.11 ms | 8.36 ms | 33 ms | 37 ms | 0 |
| Find by ID | 282,721 | 9,000 | 398 µs | 185 µs | 4.30 ms | 5.49 ms | 6.33 ms | 8.13 ms | 121,110 |
| Find earliest expiry | 31,413 | 1,000 | 398 µs | 179 µs | 4.30 ms | 5.31 ms | 6.27 ms | 7.45 ms | 0 |
| Delete by ID | 8,893 | 300 | 3.67 ms | 4.18 ms | 6.87 ms | 7.76 ms | 8.73 ms | 8.73 ms | 3,874 |
| Delete by expiry | 15,488 | 500 | 1.97 ms | 246 µs | 5.49 ms | 6.09 ms | 7.11 ms | 8.05 ms | 15,488 |
| State transition | 4,424 | 100 | 3.60 ms | 3.25 ms | 8.60 ms | 9.67 ms | 35 ms | 35 ms | 1,874 |
| Recycle storage | 1,723 | 100 | 2.00 ms | 276 µs | 5.37 ms | 6.03 ms | 6.41 ms | 6.41 ms | 1,723 |

### Mixed phase, rws run (readers READ_WRITE_SHARED)

| Operation | txns | txn/s | mean | p50 | p99 | p99.9 | p99.99 | max | misses |
|---|---|---|---|---|---|---|---|---|---|
| Insert | 39,566 | 1,300 | 2.08 ms | 2.06 ms | 4.12 ms | 27 ms | 42 ms | 45 ms | 0 |
| Find by ID | 86,228 | 2,800 | 1.30 ms | 1.27 ms | 3.70 ms | 20 ms | 43 ms | 48 ms | 36,927 |
| Find earliest expiry | 9,579 | 300 | 1.29 ms | 1.27 ms | 3.52 ms | 5.97 ms | 44 ms | 44 ms | 0 |
| Delete by ID | 19,851 | 600 | 1.17 ms | 1.03 ms | 3.67 ms | 5.14 ms | 41 ms | 41 ms | 8,475 |
| Delete by expiry | 34,279 | 1,100 | 78 µs | 66 µs | 233 µs | 358 µs | 459 µs | 926 µs | 34,279 |
| State transition | 9,852 | 300 | 1.66 ms | 2.03 ms | 4.60 ms | 5.91 ms | 35 ms | 35 ms | 4,205 |
| Recycle storage | 3,927 | 100 | 80 µs | 67 µs | 233 µs | 373 µs | 403 µs | 402 µs | 3,927 |

### Mixed phase, rws-ckpt run (async checkpoints running)

| Operation | txns | txn/s | mean | p50 | p99 | p99.9 | p99.99 | max | misses |
|---|---|---|---|---|---|---|---|---|---|
| Insert | 38,164 | 1,200 | 2.11 ms | 1.91 ms | 4.60 ms | 27 ms | 68 ms | 826 ms | 0 |
| Find by ID | 81,718 | 2,600 | 1.38 ms | 1.24 ms | 3.88 ms | 27 ms | 62 ms | 884 ms | 34,863 |
| Find earliest expiry | 9,078 | 300 | 1.39 ms | 1.24 ms | 3.73 ms | 13 ms | 831 ms | 831 ms | 0 |
| Delete by ID | 19,108 | 600 | 1.27 ms | 1.25 ms | 4.06 ms | 17 ms | 183 ms | 881 ms | 8,255 |
| Delete by expiry | 33,331 | 1,100 | 98 µs | 86 µs | 287 µs | 590 µs | 3.11 ms | 9.56 ms | 33,331 |
| State transition | 9,485 | 300 | 1.75 ms | 1.97 ms | 5.85 ms | 31 ms | 179 ms | 179 ms | 4,102 |
| Recycle storage | 3,810 | 100 | 100 µs | 89 µs | 299 µs | 545 µs | 771 µs | 771 µs | 3,810 |

Source: `Common\MemDB2\Bench\results.csv` and `results_execution_log.txt`, 10 November 2026.